

Comparação de Algoritmos de Ordenação

Trabalho Prático – Avaliação Somativa (TDE 04)

Resolução de Problemas Estruturados em Computação – Bacharelado em Sistemas de Informação

1. Introdução

Este relatório apresenta a comparação de desempenho entre três algoritmos clássicos de ordenação — Bubble Sort, Insertion Sort e Quick Sort — implementados em Java sem o uso de estruturas de dados prontas da linguagem (como `ArrayList`, `Collections` ou `Arrays.sort`), utilizando apenas vetores (arrays) manipulados manualmente.

Os algoritmos foram aplicados a nove conjuntos de dados fornecidos em arquivos CSV, divididos em três categorias (dados em ordem aleatória, dados já em ordem crescente e dados já em ordem decrescente) e três tamanhos distintos (100, 1.000 e 10.000 elementos). O objetivo é avaliar como o tamanho da entrada e a disposição inicial dos dados afetam o tempo de execução de cada algoritmo.

O tempo de execução de cada algoritmo foi medido com `System.nanoTime()`, considerando apenas o tempo da rotina de ordenação (sem incluir a leitura do arquivo). Para reduzir o impacto de ruído nas medições, cada combinação algoritmo/conjunto foi executada 5 vezes, e o valor reportado corresponde à média dessas execuções, convertida de nanossegundos para milissegundos.

2. Implementação

Os três algoritmos foram implementados utilizando exclusivamente vetores do tipo `int[]` e laços de controle nativos da linguagem (`for/while`). A leitura dos arquivos CSV foi feita com `BufferedReader`, em duas passadas: a primeira para contar o número de linhas e dimensionar o vetor, e a segunda para preencher os valores, evitando o uso de listas dinâmicas.

Bubble Sort: percorre o vetor repetidamente, comparando elementos adjacentes e trocando-os de posição quando estão fora de ordem. Foi incluída a otimização clássica de interromper a execução caso uma passagem completa não realize nenhuma troca, o que reduz drasticamente o tempo em vetores já ordenados.

Insertion Sort: constrói a porção ordenada do vetor da esquerda para a direita, deslocando elementos maiores para a direita até encontrar a posição correta do elemento atual. É naturalmente eficiente quando os dados já estão ordenados ou quase ordenados.

Quick Sort: implementado de forma recursiva, utilizando o esquema de partição de Lomuto, no qual o **último elemento de cada partição é escolhido como pivô**, conforme especificado no enunciado. Os elementos menores ou iguais ao pivô são movidos para a esquerda e os maiores para a direita, e o processo é repetido recursivamente para as duas sub-partições.

3. Resultados

A tabela abaixo apresenta o tempo médio de execução (em milissegundos) de cada algoritmo para cada um dos nove conjuntos de dados.

Conjunto de Dados	Tamanho	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório	100	0,1634	0,0575	0,0248
Aleatório	1.000	3,2710	2,8200	0,3376
Aleatório	10.000	127,1665	9,1402	2,2552
Crescente	100	0,0003	0,0003	0,0160
Crescente	1.000	0,0062	0,0011	2,1734
Crescente	10.000	0,0041	0,0093	77,5059
Decrescente	100	0,1174	0,0024	0,0172
Decrescente	1.000	1,3798	0,8628	0,3928
Decrescente	10.000	64,1719	16,9592	37,8167

O gráfico abaixo compara visualmente os tempos de execução (escala logarítmica) dos três algoritmos para cada categoria de conjunto de dados e cada tamanho.

Comparação de Algoritmos de Ordenação - Tempo de Execução (ms)

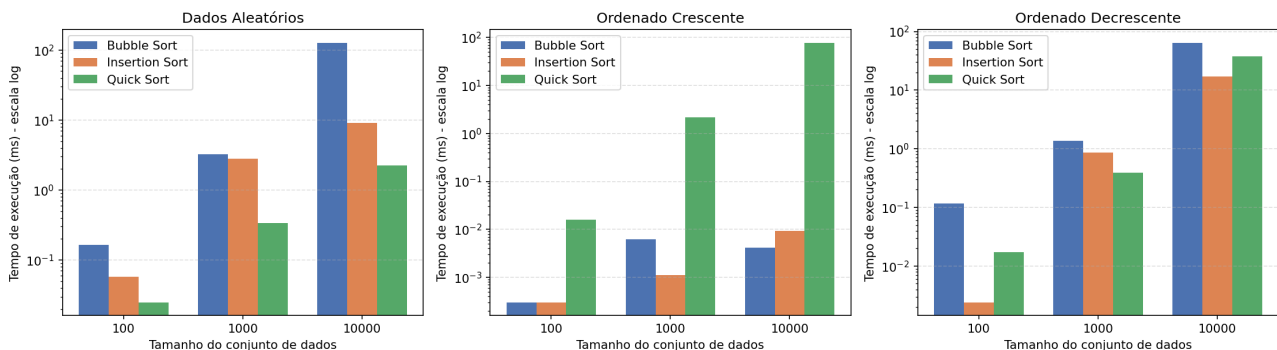


Figura 1 – Tempo médio de execução (ms, escala logarítmica) por algoritmo, categoria e tamanho do conjunto de dados.

4. Análise dos Resultados

4.1 Dados em ordem aleatória

Para os conjuntos aleatórios, o Quick Sort foi o algoritmo mais eficiente em todos os tamanhos, especialmente em 10.000 elementos (2,26 ms contra 127,17 ms do Bubble Sort e 9,14 ms do Insertion Sort). Isso ocorre porque, em dados aleatórios, o pivô (último elemento) tende a produzir partições razoavelmente equilibradas na maioria das chamadas recursivas, fazendo o Quick Sort se comportar próximo do seu caso médio $O(n \log n)$. O Bubble Sort, por outro lado, apresenta o pior desempenho, já que sua complexidade $O(n^2)$ com muitas trocas é evidenciada quando não há nenhuma ordem prévia nos dados.

4.2 Dados já ordenados em ordem crescente

Neste cenário, Bubble Sort e Insertion Sort tiveram tempos extremamente baixos, próximos de zero, mesmo para 10.000 elementos (0,0041 ms e 0,0093 ms, respectivamente). Isso é esperado: o Insertion Sort percorre o vetor sem nunca precisar deslocar elementos (cada novo elemento já está na posição correta), e o Bubble Sort, com a otimização de parar quando não há trocas, identifica que o vetor já está ordenado já na primeira passagem.

O comportamento do Quick Sort, entretanto, foi exatamente o oposto: ele apresentou o pior tempo entre os três algoritmos para 1.000 e 10.000 elementos, chegando a 77,51 ms para 10.000 elementos. Esse resultado ilustra o **pior caso conhecido do Quick Sort**: ao escolher sempre o último elemento como pivô em um vetor já crescente, esse pivô é sempre o *maior valor* da partição atual. Como consequência, cada partição gera um lado vazio e outro com $n-1$ elementos, degenerando a recursão para $O(n^2)$ e fazendo a profundidade da recursão ser igual a n .

4.3 Dados já ordenados em ordem decrescente

Para dados em ordem decrescente, o Insertion Sort apresentou o melhor desempenho geral (16,96 ms para 10.000 elementos), pois embora cada novo elemento precise ser deslocado até o início do vetor (gerando $O(n^2)$ comparações/deslocamentos), as operações são simples (deslocamentos de inteiros), o que ainda é mais rápido que as trocas completas do Bubble Sort.

O Bubble Sort apresentou desempenho intermediário/alto (64,17 ms para 10.000 elementos): como o vetor está na ordem inversa da desejada, todo elemento precisa "subir" até sua posição correta, e cada passagem realiza o número máximo possível de trocas, resultando em um comportamento $O(n^2)$ sem qualquer ganho da otimização de parada antecipada.

Já o Quick Sort, novamente com o pivô no último elemento, sofre o mesmo problema do caso crescente: em um vetor decrescente, o último elemento é sempre o *menor valor* da partição, o que também gera partições extremamente desbalanceadas (uma vazia e outra com $n-1$ elementos), levando a um tempo de 37,82 ms para 10.000 elementos — pior que o Insertion Sort, embora melhor que no caso crescente, já que a constante de comparação por elemento é menor que a do Bubble Sort.

4.4 Conclusão Geral

Os resultados confirmam que **não existe um algoritmo universalmente melhor**: o Quick Sort com pivô fixo no último elemento é altamente eficiente para dados aleatórios, mas se degrada para o pior caso $O(n^2)$ quando os dados já estão ordenados (crescente ou decrescente), exatamente as situações em que Insertion Sort (e, em menor grau, Bubble Sort com a otimização de parada) se

tornam extremamente rápidos. Esse comportamento reforça a importância da escolha do pivô: estratégias como pivô aleatório, mediana de três ou mediana-de-medianas evitariam o pior caso do Quick Sort justamente nos cenários onde os dados já apresentam alguma ordenação prévia.

Em termos gerais, para conjuntos grandes e sem informação prévia sobre a ordenação dos dados, o Quick Sort permanece a melhor escolha em média, mas o Insertion Sort se destaca como opção robusta para dados parcial ou totalmente ordenados, e o Bubble Sort, mesmo com a otimização de parada antecipada, deve ser evitado para conjuntos grandes e desordenados devido à sua complexidade $O(n^2)$ sem mitigação eficaz.

Observação: os tempos medidos podem variar entre execuções devido a fatores externos à JVM (carga do sistema, JIT warm-up, garbage collector), porém a média de 5 execuções utilizada neste relatório foi suficiente para evidenciar de forma consistente as tendências assintóticas discutidas acima.